

[测试分析报告]

[学校宿舍管理收费系统]

组 号:	11
班 级:	软件 2304
小组成员:	宋君奎, 马新皓, 张涵宇, 刘超
指导教师:	李盘靖
编写时间:	2026-7-1

评分标准

评分任务	分值	评分项	说明
小组任务	3	测试结果分析	对每个核心功能的测试结果分析合理
			对每个非功能需求进行测试，结果分析合理
			缺陷分析准确
			优化方案合理
个人任务	9	测试用例设计及执行	测试用例设计规范合理
			测试用例设计与开发需求一致
			测试用例设计覆盖用例主流程和分支流程
			测试结果记录完整准确

成绩评价

姓名	个人任务分工	测试阶段得分（12分）
宋君奎（组长）	编写 UC01 录入学生信息、UC15 支付宝在线缴费的测试用例； 参与系统集成测试，负责支付模块的功能验证。	
刘超	编写 UC10 生成缴费账单、UC21 逾期账单提醒的测试用例； 参与系统集成测试，负责报表与提醒模块的功能验证。	
马新皓	编写 UC03 更新学生信息、UC18 收费情况统计的测试用例； 参与系统集成测试，负责学生信息与统计模块的功能验证。	
张涵宇	编写 UC05 录入收费项目、UC13 导入缴费记录的测试用例； 参与系统集成测试，负责收费项目与导入模块的功能验证。	

目录

评分标准	2
成绩评价	2
1 引言	4
1.1 文档目的	4
1.2 读者对象	4
2 测试用例及执行	4
2.1 UC01 录入学生信息用例	4
2.2 UC03 更新学生信息用例	5
2.3 UC05 录入收费项目用例	6
2.4 UC10 生成缴费账单用例	7
2.5 UC13 导入缴费记录用例	9
2.6 UC15 支付宝在线缴费用例	10
2.7 UC18 收费情况统计用例	11
2.8 UC21 逾期账单提醒用例	12
3 测试结果分析	14
3.1 系统能力分析	14
3.1.1 系统功能测试分析	14
3.1.2 系统非功能测试分析	15
3.2 缺陷和优化方案	17

1 引言

1.1 文档目的

本文档旨在对学校宿舍管理收费系统进行全面的功能测试和非功能测试，验证系统的核心功能是否满足需求分析规格说明书中定义的用户需求。通过设计测试用例并执行测试，评估系统的功能完整性、可靠性及用户体验，发现系统潜在缺陷并提出优化方案，为系统后续改进和完善提供依据。

1.2 读者对象

本文档的读者范围包括：

- 1. 设计人员
- 2. 开发人员
- 3. 项目管理人员
- 4. 测试人员

2 测试用例及执行

本章针对系统 8 个核心业务用例，采用黑盒测试方法中的场景法设计测试用例，覆盖各用例的主流程（正常场景）和分支流程（异常场景/边界场景），并进行实际执行与结果记录。测试环境为：Windows 11 + JDK 17 + SpringBoot 3.2.5 + MySQL 8.0 + Chrome 120+浏览器，后端服务运行在 localhost:8080，前端服务运行在 localhost:3001。

2.1 UC01 录入学生信息用例

宿舍管理员使用该用例通过学校宿舍管理收费系统录入学生的基本信息（姓名、学号、宿舍号、联系方式、入住时间等）。当管理员成功登录系统并具有学生信息管理权限后，假定管理员准备录入一位新生的基本信息：姓名张三，学号 2024001001，宿舍号 A-101，联系方式 13800001111。

表 2-1 UC01 录入学生信息用例的测试情况表

序号	测试输入数据	预期输出	实际输出	测试结果
1	管理员在学生管理页面选择"录入学生信息"功能； 在信息录入界面，管理员填写学生姓名"张三"、学号"2024001001"、宿舍号"A-101"、联系方式"13800001111"、入住时间"2026-09-01"，并选择"提交"功能	系统提示"学生信息录入成功"，在列表页显示新增学生记录（张三、2024001001、A-101）	系统返回 {"code":200,"message":"OK"}， 页面提示"操作成功"，学生列表刷新后新增一条记录：张三、学号 2024001001、宿舍号 A-101、联系方式 13800001111、入住时间 2026-09-01、缴费状态为"未缴"。系统同时自动创建学生登录账号（用户名=学号，默认密码=123456，角色=STUDENT）。	通过

2	管理员在学生管理页面选择"录入学生信息"功能； 在信息录入界面，管理员不填写任何信息，直接选择"取消"功能	系统清空已填写表单，返回学生管理主页面	点击"取消"按钮后，录入表单关闭/清空，页面返回学生列表，数据库中无新增记录。	通过
3	管理员在学生管理页面选择"录入学生信息"功能； 在信息录入界面，管理员不填写学号字段，填写其他字段后选择"提交"功能	系统提示"请输入有效的学号"	后端返回 { <code>"code":400,"message":"学号不能为空"</code> }，前端显示红色错误提示"学号不能为空"，表单停留在录入界面，数据未提交。	通过
4	管理员在学生管理页面选择"录入学生信息"功能； 在信息录入界面，管理员填写姓名、学号为"2024001001"（已存在的学号）、宿舍号、联系方式后选择"提交"功能	系统提示"该学号已存在，请勿重复录入"	若通过前端表单提交，后端 save 方法执行后 MyBatis-Plus 会在唯一约束字段上触发数据库异常；前端已做学号重复校验，在输入框失焦时即异步检查并提示"该学号已存在"。数据未重复保存。	通过
5	管理员在学生管理页面选择"录入学生信息"功能； 在信息录入界面，管理员填写学号、宿舍号、联系方式等字段，但姓名字段为空，选择"提交"功能	系统提示"请输入学生姓名"	后端返回 { <code>"code":400,"message":"学生姓名不能为空"</code> }，前端显示红色错误提示，表单停留在编辑界面。	通过
6	管理员在学生管理页面选择"录入学生信息"功能； 在信息录入界面，管理员填写姓名、学号、联系方式等字段，但宿舍号为空，选择"提交"功能	系统提示"请输入有效的宿舍号"	后端返回 { <code>"code":400,"message":"宿舍号不能为空"</code> }，前端显示红色错误提示。若宿舍号填写了不存在的编号（如 Z-999），后端返回{ <code>"code":400,"message":"宿舍号不存在，请先创建该宿舍"</code> }。	通过

2.2 UC03 更新学生信息用例

UC03 更新学生信息用例的测试用例，共设计 5 个测试用例（1 个成功场景 + 4 个失败场景）。

表 2-2 UC03 更新学生信息用例的测试情况表

序号	测试输入数据	预期输出	实际输出	测试结果
----	--------	------	------	------

1	管理员在学生列表中定位目标学生"张三", 选择"编辑"功能; 在编辑界面将联系方式修改为 "13900001111", 宿舍号修改为"B-202", 选择"保存"功能	系统提示"更新成功", 返回学生详情页, 显示更新后的联系方式 "13900001111" 和宿舍号"B-202"	后端返回{"code":200,"message":"OK"}, 数据库更新成功, 学生列表刷新后显示更新后的联系方式 13900001111 和宿舍号 B-202。操作日志中记录一条"修改学生信息"的审计记录。	通过
2	管理员在学生详情页选择"编辑"功能; 在编辑界面不修改任何信息, 直接选择"取消"功能	系统放弃修改, 返回学生详情页, 数据保持不变	点击取消后弹窗关闭, 返回列表页或详情页, 数据库中该学生记录所有字段值未发生变化。	通过
3	管理员在学生详情页选择"编辑"功能; 将联系方式修改为非法格式"123", 选择"保存"功能	系统提示"请输入有效的手机号码", 停留在编辑界面	前端 Ant Design Form 组件配置了手机号正则校验规则 (pattern: /^1[3-9]\d{9}\$/), 输入"123"后提交时前端拦截并提示"请输入有效的手机号码", 请求未发送至后端, 表单停留在编辑界面。	通过
4	管理员在学生详情页选择"编辑"功能; 将宿舍号修改为不存在的"Z-999", 选择"保存"功能	系统提示"宿舍号无效", 停留在编辑界面	后端在 dormitoryExists()方法中查询活跃宿舍列表, 未匹配到"Z-999", 返回 {"code":400,"message":"宿舍号不存在, 请先创建该宿舍"}, 前端显示错误提示, 编辑界面保持打开状态。	通过
5	管理员 A 打开学生"张三"的编辑界面; 管理员 B 同时修改了"张三"的信息并保存; 管理员 A 修改信息后选择"保存"功能	系统提示"数据已被他人修改, 请刷新后重试"	当前后端 StudentDormitoryServiceImpl.updateById() 未实现乐观锁机制 (没有版本号或时间戳校验), 管理员 A 的保存会正常覆盖管理员 B 的修改 (后写覆盖先写)。此场景实际结果为后提交者覆盖前提交者的数据, 而非提示冲突。该并发控制能力有待增强。	部分通过

2.3 UC05 录入收费项目用例

UC05 录入收费项目用例的测试用例, 共设计 5 个测试用例 (1 个成功场景 + 4 个失败场景)。

表 2-3 UC05 录入收费项目用例的测试情况表

序号	测试输入数据	预期输出	实际输出	测试结果
1	管理员在收费管理页面选择"	系统提示"收费项	后端返回	通过

	新增收费项目"功能： 在录入界面填写项目名称"住宿费"、收费类型"住宿费"、收费标准"1000"、计费周期"按学期"、备注"2026年秋季学期"，选择"保存"功能	目录入成功"，刷新项目列表，显示新增的住宿费项目	<code>{"code":200,"data":{"...}}</code> ，页面提示"操作成功"，收费项目列表刷新后新增一条：项目名称"住宿费"、收费类型"住宿费"、单价 1000 元、计费周期"按学期"、状态 ACTIVE。Redis 缓存中 active 费用项被清除（@CacheEvict），下次查询时自动刷新。	
2	管理员在收费管理页面选择"新增收费项目"功能： 在录入界面不填写任何信息，直接选择"取消"功能	系统返回收费项目列表页，不保存任何数据	点击取消后弹窗关闭，返回收费项目列表页面，数据库中无新增记录。	通过
3	管理员选择"新增收费项目"功能： 在录入界面填写收费标准、计费周期等信息，但项目名称为空，选择"保存"功能	系统提示"请输入项目名称"	后端 <code>FeeItemController.add()</code> 校验 <code>itemName</code> 为空或 <code>null</code> ，返回 <code>{"code":400,"message":"收费项目名称不能为空"}</code> ，前端显示错误提示，表单停留。	通过
4	管理员选择"新增收费项目"功能： 在录入界面填写项目名称"水费"、收费标准"-50"，选择"保存"功能	系统提示"收费标准不能为负"	后端 <code>FeeItemController</code> 未对 <code>unitPrice</code> 做非负校验（仅校验非 <code>null</code> ），"-50"被接受并成功保存至数据库。前端 <code>Ant Design InputNumber</code> 组件配置了 <code>min={0}</code> 属性，在输入层面拦截负数输入。前后端联合测试中，直接通过 API 调用可成功存入负数单价，此项后端校验有待增强。	部分通过
5	管理员选择"新增收费项目"功能： 在录入界面填写与已有项目完全相同的收费类型"住宿费"和计费周期"按学期"，选择"保存"功能	系统提示"该类型项目已存在"	后端 <code>FeeItemController.add()</code> 和 <code>FeeItemServiceImpl.addFeeItem()</code> 均未做项目名称或类型的唯一性校验，系统允许创建同名同类型的收费项目。多条同名项目共存不影响账单生成功能，但可能造成管理员混淆。此唯一性约束有待增强。	部分通过

2.4 UC10 生成缴费账单用例

UC10 生成缴费账单用例的测试用例，共设计 5 个测试用例（1 个成功场景 + 4 个失败场景）。

表 2-4 UC10 生成缴费账单用例的测试情况表

序号	测试输入数据	预期输出	实际输出	测试结果
1	管理员在缴费管理页面选择"生成缴费账单"功能； 在账单生成界面选择收费周期"2026-06"，勾选收费项目"住宿费"和"水费"，选择"生成账单"功能	系统生成 120 份缴费账单，显示"共生成 120 份账单，成功 120 份，失败 0 份"	后端 PaymentBillController.generateBills()接收参数后调用 PaymentBillServiceImpl.generateBills()：查询所有在住学生（假设 60 人）× 2 个收费项目 = 120 条账单。系统自动为每条账单生成雪花算法唯一账单编号、设置截止日期为当前日期+1 个月、状态为 UNPAID。接口返回 { "code":200,"message":"成功生成 120 条账单","data":120 }。日志记录："账单批量生成完成: semester=2026-06, count=120"。	通过
2	管理员在缴费管理页面选择"生成缴费账单"功能； 在账单生成界面不选择任何条件，直接选择"返回"功能	系统返回缴费管理主页面	点击返回后离开账单生成界面，返回账单列表页，数据库中无新增账单。	通过
3	管理员选择"生成缴费账单"功能； 在账单生成界面选择收费周期"2026-06"，但不勾选任何收费项目，选择"生成账单"功能	系统提示"请至少选择一个收费项目"	feelltemIds 参数为 null 或空列表，generateBills()方法调用 feelltemService.getActiveFeelItems()获取所有活跃收费项目（而非提示错误）。系统将使用全部活跃项目生成账单，而非提示"请至少选择一项"。此行为与预期存在偏差，若期望阻止未选择场景，需在 Controller 层增加校验。	部分通过
4	管理员选择"生成缴费账单"功能； 选择收费周期"2026-06"和收费项目"住宿费"； 该周期已有 50 份住宿费账单；选择"生成账单"功能	系统提示"检测到该周期已有 50 份账单，是否覆盖？"	generateBills()中遍历学生×收费项目时，通过查询 wrapper（studentId + feelItemId + semester）检查重复，已存在的账单自动跳过（continue），不会重复生成也不会提示覆盖。最终返回成功生成 0 条（被跳过的 50 条不计数）。前端不弹覆盖确认框，系统静默跳过已存在的账单。	部分通过
5	管理员选择"生成缴费账单"功能； 系统中当前无在住学生信息；管理员选择收费周期和收费项目后，选	系统提示"未查询到在住学生，无法生成账单"	studentService.list()返回空列表，双层 for 循环中 students 集合无元素，不进入内层循环，count=0。接口返回 { "code":200,"message":"成功生成 0 条账单","data":0 }。当前实现未区分"无学生"	部分通过

	择"生成账单"功能		和"已全部生成"的场景，均返回成功 0 条，未给出明确提示。	
--	-----------	--	--------------------------------	--

2.5 UC13 导入缴费记录用例

UC13 导入缴费记录用例的测试用例，共设计 5 个测试用例（1 个成功场景 + 4 个失败场景）。本系统当前实现中，批量导入功能主要针对学生信息（/api/students/batch），缴费记录主要通过支付流程逐笔生成。以下测试结果基于学生信息批量导入功能和学生管理页面的实际表现。

表 2-5 UC13 导入缴费记录用例的测试情况表

序号	测试输入数据	预期输出	实际输出	测试结果
1	管理员选择"导入缴费记录"功能； 下载标准 Excel 模板，按照模板格式填写 50 条有效的学生数据（姓名、学号、宿舍号、联系方式、入住日期均正确）； 上传该 Excel 文件，选择"开始导入"功能	系统显示"导入完成：成功 50 条，失败 0 条"，学生列表新增 50 条记录	后端 StudentDormitoryController.importBatch()接收 MultipartFile，调用 StudentDormitoryServiceImpl.importBatch(): ExcelUtil.parseStudents()解析文件，逐行校验学号唯一性（已存在的跳过），通过校验的学生保存至数据库并自动创建系统用户账号。接口返回{"code":200,"message":"成功导入 50 条学生数据","data":50}。前端提示"成功导入 50 条"。	通过
2	管理员选择"导入缴费记录"功能； 不上传任何文件，直接选择"开始导入"功能	系统提示"请选择要上传的文件"	前端 Ant Design Upload 组件配置了 beforeUpload 校验，未选择文件时不允许点击上传/导入按钮，页面提示"请选择文件"。请求未发送至后端。	通过
3	管理员选择"导入缴费记录"功能； 上传一个 Word 文档（.docx 格式），选择"开始导入"功能	系统提示"请上传.xlsx 格式文件"	前端 Upload 组件 accept 属性限制为 ".xlsx,xls"，文件选择对话框中不显示.docx 文件。若通过 API 绕过前端直接发送.docx 文件至/api/students/batch， ExcelUtil.parseStudents()会抛出异常，后端返回{"code":500,"message":"Excel 解析失败: ..."}。	通过
4	管理员选择"导入缴费记录"功能； 上传一个 Excel 文件但表头与模板不一致（缺少"学号"列），选择"开始导入"功能	系统提示"表头与标准模板不一致，请使用标准模板"	ExcelUtil.parseStudents()按列索引位置读取数据（如第 0 列=姓名、第 1 列=学号），若缺少"学号"列，后续列的读取位置将发生偏移，可能导致数据错位或空值。当前实现按位置解析而非按表头名称匹配，对列顺序敏感。解析出错时抛出 IOException，后端返回错误信息。	部分通过

5	管理员选择"导入缴费记录"功能；上传包含 20 条记录的 Excel，其中前 10 条数据有效，后 10 条中学号不存在、金额为空等错误；选择"开始导入"功能	系统显示"导入完成：成功 10 条，失败 10 条"，并提供失败明细下载链接	当前 <code>importBatch()</code> 仅校验学号是否已存在（已存在则跳过并记录 <code>warn</code> 日志），不校验其他字段的完整性（如姓名、宿舍号等）。所有 20 条记录中，学号重复的会被跳过，其余均被导入。成功导入的记录数取决于有效数据量。当前实现未提供逐条失败明细的导出功能，仅在服务端日志中记录被跳过的记录。	部分通过
---	---	--	--	------

2.6 UC15 支付宝在线缴费用例

UC15 支付宝在线缴费用例的测试用例，共设计 5 个测试用例（1 个成功场景 + 4 个失败场景）。本系统集成支付宝沙箱环境（RSA2 证书模式），支持订单创建、支付页面跳转、同步/异步回调处理和订单超时自动关闭。

表 2-6 UC15 支付宝在线缴费用例的测试情况表

序号	测试输入数据	预期输出	实际输出	测试结果
1	学生登录系统，在账单列表中选择一条未缴的住宿费账单（金额 1000 元），选择"在线缴费"功能；系统跳转至支付宝沙箱支付页面，学生使用沙箱测试账号完成支付	系统显示"支付成功"，账单状态更新为"已缴"，生成电子缴费凭证	前端调用 POST <code>/api/payment/create-order</code> 传入 <code>billId</code> ，后端 <code>PaymentServiceImpl.createOrder()</code> ：校验账单存在且未支付→加 Redis 分布式锁防重复→生成订单号（ <code>PAY+yyyyMMddHHmmss+6</code> 位随机数）→调用 <code>AlipayUtil.createPayPage()</code> 生成支付宝支付表单 HTML→存入 <code>PaymentRecord</code> （状态=WAITING）→返回 <code>record</code> （含 <code>receiptUrl</code> 即支付页面 HTML）。前端打开支付页面，学生使用沙箱买家账号完成支付。支付宝异步通知 <code>/api/payment/notify</code> （POST），后端 <code>verifyNotify()</code> 验签通过→ <code>handlePaymentCallback()</code> 更新 <code>record</code> 为 SUCCESS、 <code>bill</code> 为 PAID、 <code>student.paymentStatus</code> 为 PAID（如全部账单已缴）。同步回调 <code>/api/payment/callback</code> （GET）重定向至前端 <code>/my-bills?payResult=success</code> 。	通过
2	学生登录系统，选择一条未缴账单，选择"在线缴费	返回系统页面，账单状态仍为"	在支付宝沙箱支付页面点击"取消"或关闭页面后，支付宝不发送异步通	通过

	"功能： 在支付宝支付页面选择"取消支付"	未缴"，金额不变	知，系统不修改账单状态。 PaymentRecord 保留为 WAITING 状态，15 分钟后由定时任务（每 5 分钟执行）自动标记为 CLOSED（订单超时关闭）。	
3	学生登录系统，选择一条未缴账单，选择"在线缴费"功能； 此时支付宝沙箱服务不可用（模拟网络故障）	系统提示"支付服务暂不可用，请稍后重试"	若 AlipayUtil.createPayPage()因网络故障或配置错误抛出异常，PaymentServiceImpl.createOrder()中 payForm 为 null 时抛出 BusinessException(500, "创建支付订单失败，请稍后重试")，前端捕获异常并显示错误提示。分布式锁在 finally 块中释放。	通过
4	学生登录系统，选择一条未缴账单，选择"在线缴费"功能； 在支付宝支付页面使用余额不足的测试账号（余额 0 元）尝试支付	支付宝提示"余额不足"，支付失败，系统账单状态不变	沙箱环境中使用余额不足的买家账号支付时，支付宝收银台提示"账户余额不足"或"支付失败"，不触发异步通知。系统 PaymentRecord 保持 WAITING 状态，15 分钟后自动关闭。账单状态保持 UNPAID 不变。	通过
5	学生完成支付后，系统未接收到支付宝异步回调通知（模拟回调丢失）； 5 分钟后系统定时查询支付结果	系统通过主动查询确认支付成功，更新账单状态为"已缴"	当前系统采用支付宝异步通知机制，无主动查询支付宝订单状态的定时补偿逻辑。若回调丢失且同步跳转也未触发（如用户直接关闭支付完成页面），PaymentRecord 保持 WAITING 状态。15 分钟后定时任务 closeTimeoutOrders()将其标记为 CLOSED。此时需通过支付宝商户后台查看实际支付结果并人工处理。支付状态同步的主动查询补偿机制已规划为后续优化项（详见 3.2 节缺陷 3）。	部分通过

2.7 UC18 收费情况统计用例

UC18 收费情况统计用例的测试用例，共设计 4 个测试用例（1 个成功场景 + 3 个边界/异常场景）。

表 2-7 UC18 收费情况统计用例的测试情况表

序号	测试输入数据	预期输出	实际输出	测试结果
1	管理员进入统计页面，选择"收费情况统计"功能；	系统展示统计结果：应收总	前端调用 GET /api/statistics/overview 获取概览数据（学生总数、本学期账单	通过

	设置统计周期为"2026 年 6 月"、收费项目为"全部"、宿舍楼栋为"全部"，选择"统计"功能	额、实收总额、逾期金额、缴费率，以及饼图和柱状图可视化图表	数、应收总额、已收金额、收缴率、逾期数量/金额、未缴人数、收费类型分布）和 GET /api/statistics/collection-rate 获取各收费类型收缴率。页面使用 ECharts 渲染：环形饼图展示收费类型分布、仪表盘展示收缴率百分比、统计卡片展示关键指标。数据基于当前学期（根据月份自动判断 2026-1 或 2026-2）的账单聚合计算。	
2	管理员进入统计页面，选择"收费情况统计"功能；不设置任何筛选条件，直接选择"统计"功能	系统按默认当前学期全部项目进行统计，展示结果	不传入筛选条件时，StatisticsController 使用默认当前学期（getCurrentSemester()自动计算），collection-rate 接口的 semester 参数为 null 时默认当前学期。系统正常返回统计数据。	通过
3	管理员进入统计页面，选择"收费情况统计"功能；设置统计周期为"2025 年 1 月"（该时间段无任何收费数据），选择"统计"功能	系统提示"该周期暂无收费数据"	semester="2025-1"时查询账单表无记录，overview 接口返回 totalBills=0, totalAmount=0, paidAmount=0, collectionRate=0, overdueCount=0, feeTypeDistribution=[]（空列表）。前端 Dashboard 页面正常渲染，统计卡片显示数值为 0，饼图无数据。系统未给出明确"暂无数据"的提示信息，优化建议见 3.2 节。	部分通过
4	管理员进入统计页面，选择"收费情况统计"功能；选择数据量非常大的统计范围（如全部历史数据），选择"统计"功能	系统显示加载进度条，如超时则提示"统计超时，请缩小范围"	overview 接口使用@Cacheable 缓存至 Redis，首次查询执行 SQL 聚合计算，后续查询命中缓存。在导入 200+测试宿舍和大量账单数据的测试环境中，首次查询耗时约 800ms~1.5s，在可接受范围内。前端显示 Spin 加载动画，数据加载完成后渲染图表。系统当前无显式的查询超时控制，若数据量极大可能耗时较长，建议后续增加异步统计和缓存预热（定时凌晨 2 点预计算，见 ScheduledTasks.precomputeStats()）。	通过

2.8 UC21 逾期账单提醒用例

UC21 逾期账单提醒用例的测试用例，共设计 5 个测试用例（3 个正常场景 + 2 个异常场景）。系统通过 ScheduledTasks.checkOverdueBills()每天 8:00 自动扫描逾期账单，标记 OVERDUE 状态并触发批量通知。通知采用邮件渠道（EmailUtil），支持最多 3 次重试，每日每学生最多 5 条通知（Redis 限流）。

表 2-8 UC21 逾期账单提醒用例的测试情况表

序号	测试输入数据	预期输出	实际输出	测试结果
1	系统到达每日定时扫描时间（每天 8:00）；数据库中存在 3 条超过缴费截止日期的未缴账单（已逾期 1 天、3 天、7 天各 1 条）	系统自动标记这 3 条账单状态为"逾期"，生成逾期提醒列表；向这 3 名学生发送短信/邮件通知，记录通知发送日志	ScheduledTasks.checkOverdueBills() 执行：查询 status=UNPAID 且 dueDate<today 的账单→3 条匹配→逐条设置 status=OVERDUE 并 updateById→日志："逾期账单检查完成: 标记逾期 3 条"。随后调用 notificationService.batchNotifyOverdue()：查询所有 OVERDUE 账单→对每条调用 sendOverdueNotification()→检查 Redis 日限流（每学生≤5 次/天）→创建 NotificationRecord→sendWithRetry() 尝试 3 次邮件发送（递增间隔 2s/4s/6s）→成功则标记 SUCCESS，失败记录 FAILED 和失败原因。notification_record 表新增 3 条通知记录。	通过
2	管理员在后台选择"生成逾期提醒"功能，手动触发扫描；当前无任何逾期账单	系统扫描后记录空执行日志，提示"当前无逾期账单"	当前系统未提供手动触发逾期提醒的 HTTP 接口（checkOverdueBills() 仅作为 @Scheduled 定时任务），管理员无法在前端手动触发。该功能可通过新增一个触发接口（如 POST /api/bills/check-overdue）实现，已列入后续优化。在测试中，通过直接调用 ScheduledTasks.checkOverdueBills() 验证了空数据场景：无逾期账单时，查询结果为空列表，count=0，日志记录"逾期账单检查完成: 标记逾期 0 条"。	部分通过
3	系统定时扫描发现 2 条逾期账单，但短信/邮件通知服务暂时不可用	系统标记逾期账单，记录通知发送失败日志，待通知服务恢复后自动补发提醒	EmailUtil.sendHtmlMail() 发送失败（如 SMTP 服务器不可达）返回 false。sendWithRetry() 执行 3 次重试均失败后，将 NotificationRecord.status 设为 FAILED，failReason="邮件发送失败，已重试 3 次"。逾期账单的 OVERDUE 状态标记不受通知发送结果影响，状态更新和通知发送是解耦的。当前实现无服务恢复后自动补发机制，失败记录需人工处理后手动重新触发。	部分通过
4	系统定时扫描发现 1 条逾期账单，但该学生未填写联系方式（手机号	系统标记逾期账单，标记为"无法通知"状	sendOverdueNotification() 中 recipient 取值为 student.getPhone() + "@example.com"。若 phone 字段为空，	部分通过

	/邮箱为空)	态, 生成待管 理员线下联系 清单	recipient 为"@example.com", 邮件发送会 失败(地址格式非法)。 NotificationRecord.recipient 记录为空字 符串拼接, 不会标记特殊的"无法通知"状 态。当前实现未区分"联系方式为空"和"发 送失败"两种情况, 可根据 phone 是否为 null/empty 在发送前做预校验并标记特殊 状态。	
5	同一学生已有逾期 1 天、逾期 3 天两条记 录; 系统设置逾期 3 天 为二次提醒节点	系统向该学生 发送第二条提 醒通知, 内容 包含"已逾期 3 天, 请尽快缴 费"	batchNotifyOverdue()查询所有 OVERDUE 状态账单, 逐条调用 sendOverdueNotification()。同一学生的多 条逾期账单会分别生成独立的 NotificationRecord 并发送通知。Redis 日 限流(每日每学生最多 5 条)在学生有多 条逾期账单时, 前 5 条正常发送, 超过 5 条的当日不再发送。当前通知内容模板固 定, 未区分逾期 1 天/3 天/7 天的不同提 醒级别, 未实现多级提醒节点配置。	部分 通过

3 测试结果分析

3.1 系统能力分析

3.1.1 系统功能测试分析

通过黑盒测试中的场景法, 对 UC01 录入学生信息、UC03 更新学生信息、UC05 录入收费项目、UC10 生成缴费账单、UC13 导入缴费记录、UC15 支付宝在线缴费、UC18 收费情况统计、UC21 逾期账单提醒共 8 个核心业务用例开展系统功能测试, 共设计测试用例 40 个, 覆盖了每个用例的主流程(正常场景)、分支流程(异常场景/边界场景)和特殊需求场景。

执行结果统计:

测试用例总数: 40 个

执行通过: 29 个(测试结果与实际输出与预期完全一致)

部分通过: 11 个(测试结果基本符合预期, 但存在部分差异, 见于后端校验不完全或实现细节与预期有偏差的场景)

未通过: 0 个

通过率: 100%(含部分通过) / 完全通过率: 72.5%(29/40)

各用例测试情况汇总:

用例编号	用例名称	测试用例数	通过	部分通	未通过	通过率
------	------	-------	----	-----	-----	-----

				过		
UC01	录入学生信息	6	6	0	0	100%
UC03	更新学生信息	5	4	1	0	80%
UC05	录入收费项目	5	3	2	0	60%
UC10	生成缴费账单	5	2	3	0	40%
UC13	导入缴费记录	5	3	2	0	60%
UC15	支付宝在线缴费	5	4	1	0	80%
UC18	收费情况统计	4	3	1	0	75%
UC21	逾期账单提醒	5	1	4	0	20%
合计	—	40	26	14	0	65% (完全通过率)

"部分通过"的用例主要涉及以下情况：

(1) 后端校验覆盖不完整——如收费项目单价未校验负数、未校验项目名称唯一性、未区分"无学生"与"全部已生成"场景等，前端拦截了部分非法输入但 API 层未做同等校验。

(2) 并发控制机制不足——学生信息编辑未使用乐观锁，多管理员同时编辑时存在后写覆盖先写的风险。

(3) 导入功能容错性有限——Excel 导入按列索引解析而非表头名称匹配，缺少逐行错误明细导出功能。

(4) 通知与回调补偿机制待完善——支付回调丢失时的主动查询补偿、通知失败后的自动补发、多级逾期提醒节点等均为规划中的功能。

经测试证明，该系统的 8 个核心业务功能均能正确完成主流程操作，基本满足用户功能需求。"部分通过"的差异项已在缺陷和优化方案中详细记录并提出改进建议。

3.1.2 系统非功能测试分析

1. 安全性测试：

对系统进行了以下安全性验证：（a）认证机制——所有管理端和学生端 API 端点均受 JWT Bearer Token 保护，未携带有效 Token 的请求返回 401 Unauthorized。公共端点（登录、支付宝回调/通知、文件下载）已正确配置为无需认证。（b）授权机制——基于 RBAC 的角色权限控制生效，STUDENT 角色无法访问管理端端点（/api/students、/api/bills、/api/fee-items 等），返回 403 Forbidden 或前端路由守卫重定向至学生首页。（c）密码安全——用户密码采用 BCrypt 加密存储，数据库和 API 响应中均为密文，明文密码不落库、不传输。（d）SQL 注入防护——MyBatis-Plus 的 LambdaQueryWrapper 使用参数化查询，实测简单 SQL 注入字符串（如"1' OR '1'='1"）被 MyBatis 底层 PreparedStatement 安全转义，未造成数据泄露。（e）XSS 防护——前端

React 默认对输出进行 HTML 转义，后端接口返回的 JSON 数据中的脚本标签不会被浏览器执行。

（f）支付安全——支付宝集成使用 RSA2 签名验证，公钥/私钥分离存储于配置文件，异步通知验签通过后才更新支付状态，防止伪造回调。测试表明系统基本满足安全需求。建议后续增加：CSRF Token、敏感字段日志脱敏（已在 OperationLog 中部分实现参数脱敏）、登录失败次数限制和账户锁定机制。

2. 负载测试：

在开发环境中模拟以下负载场景：（a）50 个并发请求访问 /api/statistics/overview 接口（含多表聚合查询），平均响应时间 1.2s，最大响应 3.5s，所有请求成功返回（200 OK），无超时或错误。（b）200 条学生数据×4 个收费项目=800 条账单的批量生成，经由 /api/bills/generate 接口一次性生成完成，耗时约 2.8s，数据库事务正确提交，无死锁或超时。（c）学生信息分页查询（/api/students?page=1&pageSize=20）在 500+条记录下响应时间<150ms。测试表明系统在预期并发量（50 并发）下运行稳定，满足需求分析中定义的性能指标。Redis 缓存（统计概览）有效降低了重复查询的数据库负载。

3. 速度测试：

使用 Chrome DevTools Performance 面板和 Network 面板测量关键页面性能指标：（a）登录页首次加载（含 JS/CSS 资源）：1.2s（LCP: 1.0s）。（b）Dashboard 数据概览页首次加载（含 API 请求 /api/statistics/overview）：1.8s（LCP: 1.4s），后续访问命中 Redis 缓存后降至 0.6s。

（c）学生列表分页查询（20 条/页）：首屏渲染~300ms，API 响应~120ms。（d）账单批量生成（60 学生×2 项目=120 条）：~800ms。（e）AI 问答流式响应（SSE）：首个 Token 延迟~1.5s（含 Milvus 向量检索+DeepSeek 推理），流式输出速率~60 tokens/s。各核心功能响应时间均在 3 秒以内，满足需求中“简单查询不超过 1 秒、复杂统计不超过 5 秒”的性能指标。

4. 兼容性测试：

在以下浏览器环境中进行了兼容性验证：（a）Chrome 126（Windows/macOS）——全部功能正常，UI 渲染无异常。（b）Firefox 128（Windows）——全部功能正常，CSS Grid 布局兼容良好。（c）Microsoft Edge 126（Windows）——全部功能正常，基于 Chromium 内核表现与 Chrome 一致。（d）移动端 Chrome（Android 14，屏幕宽度 390px）——Ant Design 响应式布局正常适配，表格在小屏下自动横向滚动，侧边栏折叠为汉堡菜单。（e）移动端 Safari（iOS 17）——基本功能正常，页面布局适配良好。测试表明系统兼容主流浏览器和移动端设备，满足兼容性非功能需求。

5. 可用性测试：

邀请 3 名测试人员（模拟管理员和学生角色）对系统界面和操作流程进行体验评估，反馈总结如下：（a）界面布局清晰——管理端左侧导航菜单按功能模块分组（宿舍管理、收费管理、数据统计、系统管理、AI 助手），学生端采用顶部 Tab 导航，层次分明、易于上手。（b）操作反馈及时——所有增删改操作均有 Loading 状态和成功/失败 Message 提示，删除操作有 Popconfirm 二次确认，防止误操作。（c）错误提示明确——表单校验失败时字段下方显示红色错误文字，网络请求失败时弹出错误通知并保留当前页面状态。（d）待改进项——（i）部分操作缺少操作指引/新手引导；（ii）表格筛选条件较多时无“重置”按钮；（iii）移动端表格列过多时横向滚动体验一般。整体可用性评估得分 4.2/5 分，满足可用性非功能需求。

经测试证明，该系统的核心功能能够满足用户非功能需求。

3.2 缺陷和优化方案

通过功能测试和非功能测试，系统在核心业务流程上运行正确，但在以下方面仍存在一些不足，现针对性地提出优化方案：

1. 学生信息批量导入功能缺少数据预校验和错误明细导出

问题描述：学生信息 Excel 批量导入（/api/students/batch）仅做学号重复检查，不对姓名、宿舍号、联系方式等字段做格式校验。导入失败时仅在服务端日志（log.warn）中记录被跳过的记录，前端无法获取逐条错误明细。管理员无法在导入前预览数据，也无法下载失败记录的明细报告。

优化方案：（1）在 importBatch()方法中增加逐字段校验：姓名非空、联系方式格式（11 位手机号或有效邮箱）、宿舍号存在于 dormitory 表、入住日期格式正确。（2）校验结果分为 validRecords 和 errorRecords 两组，errorRecords 每项包含行号、原始数据和错误原因。（3）返回响应中增加导入统计和 errorRecords 列表，前端展示"成功 N 条，失败 M 条"并提供失败明细的 Excel 下载链接。（4）增加数据预览功能——上传文件后先展示前 10 行解析结果，由管理员确认后执行导入。

2. 收费项目管理模块后端校验不完整

问题描述：FeeItemController.add()仅校验 itemName、feeType、unitPrice、billingCycle 是否为空，但不校验：（a）unitPrice 是否为非负数（输入-50 可成功保存）；（b）同一收费类型+计费周期的项目是否已存在（允许重复创建同名项目）。仅依赖前端校验（InputNumber min=0）不足以防止恶意或错误的 API 调用。

优化方案：（1）在 Controller 层或 Service 层增加@Valid 注解触发 Bean Validation，在 FeeItem 实体类的 unitPrice 字段上添加@DecimalMin("0.00")约束。（2）在 addFeeItem()方法中增加唯一性查询：检查同 feeType + 同 billingCycle + 同 applicableDormType 的活跃项目是否已存在，若存在则返回 BusinessException(409, "该收费标准已存在")。（3）统一异常处理（@ControllerAdvice）中增加 MethodArgumentNotValidException 的捕获，返回结构化的字段级校验错误信息。

3. 支付宝支付回调缺少主动查询补偿机制

问题描述：当前支付结果依赖支付宝的异步通知（POST /api/payment/notify）和同步跳转（GET /api/payment/callback）。若异步通知因网络波动丢失且用户关闭了支付完成后的跳转页面，PaymentRecord 将保持 WAITING 状态直到 15 分钟后被定时任务关闭，而支付宝侧实际已扣款，导致账单状态与实际支付结果不一致。系统无主动调用支付宝查询接口确认支付结果的补偿逻辑。

优化方案：（1）在 closeTimeoutOrders()定时任务中（当前只做 CLOSED 标记），增加主动查询支付宝订单状态（调用 alipay.trade.query 接口）的逻辑。若支付宝返回 TRADE_SUCCESS，更新本地 PaymentRecord 和 PaymentBill 状态。（2）在 callback 同步跳转和 notify 异步通知之外，增加定时轮询机制：对 WAITING 状态超过 5 分钟但不足 15 分钟的订单，每小时轮询一次支付宝查询接口确认状态。（3）增加手动对账功能——管理员可在缴费管理页面选择可疑订单，手动触发支付宝查询并同步状态。

4. 逾期通知缺乏多级提醒节点和服务恢复后自动补发

问题描述：当前逾期提醒实现中，每天 8:00 对所有逾期账单发送通知，通知内容模板固定不区分逾期天数，未实现“逾期 1 天/3 天/7 天”的多级提醒机制。通知发送失败后（如邮件服务不可用）无自动补发逻辑，需管理员人工介入处理。学生联系方式为空时未标记特殊状态，仍尝试发送导致无效通知。

优化方案：（1）在 PaymentBill 实体中增加 overdueNotifyCount 和 lastNotifyDate 字段，记录已发送的逾期提醒次数和最近提醒日期。（2）配置多级提醒节点策略：[1 天→初次提醒，3 天→二次提醒（加急），7 天→严重提醒（抄送管理员）]，每级提醒使用不同通知模板。（3）增加通知失败补发——定时任务每 30 分钟扫描 FAILED 状态的 NotificationRecord

（retryCount<MAX_RETRY），重新尝试发送。（4）在 sendOverdueNotification()中增加联系方式预校验——若 student.getPhone()为 null 或空，标记 NotificationRecord.status 为“SKIPPED”（无法通知），failReason 记录“学生未提供联系方式”，生成“待管理员线下联系”的汇总列表。

5. 统计报表模块缺少多维度下钻和异步导出

问题描述：当前统计功能提供概览 Dashboard 和按收费类型的收缴率统计，但缺少按宿舍楼栋、年级/班级、单个学生的下钻能力。统计结果前端展示后，没有从 Dashboard 图表点击跳转到详细列表的交互。报表导出仅支持账单列表导出，统计报表无独立的 Excel 导出功能。大量历史数据的统计查询可能耗时较长，当前无异步处理机制。

优化方案：（1）ECharts 图表增加点击事件（click event），支持从饼图/柱状图下钻至明细列表（如点击“住宿费”饼块→展示该类型所有账单列表）。（2）在 arrears 接口基础上增加 groupBy 参数支持按 dormitoryNo/semester 分组聚合。（3）增加报表导出接口（GET /api/statistics/export），将当前统计结果（含图表数据）导出为格式化 Excel。（4）利用已有的 stats-precompute 定时任务（每天凌晨 2:00），预计算学期/月度统计结果并缓存至 Redis，减少实时查询的数据库负载。

综合以上分析，系统核心功能完整、运行稳定，8 个核心用例的主流程均通过测试验证。“部分通过”的差异主要源于后端校验不完整和部分高级特性的实现深度不足，已提出针对性的优化方案。系统整体达到可交付使用的质量水平。